Analog quantum devices for optimization problems

DuQuantum 2026: Pasqal's Challenge

Volya Okrut[1] *Emmanuel E. Umukoro*[2] *James Kingdon*[3]

Neutral Groundstate
Department of Mathematics, Duke University[1]
Department of Mathematics and Physics, North Carolina Central University[2]
Department of Computer Science, North Carolina State University[3]

October 25, 2026

Can capacity constraints be encoded without spending scarce hardware resources on slack variables?

Problem: Capacity-Constrained Clustering

Given:

- N customers with demands d_j;
- K identical vehicles, each with capacity Q;
- Pairwise customer distances d_{ij}.

Define the binary assignment variables

$$x_{jk} = \begin{cases} 1, & \text{if customer } j \text{ is assigned to vehicle } k, \\ 0, & \text{otherwise.} \end{cases}$$

Constraints

Each customer must be assigned exactly once: $\sum_{k=1}^{K} x_{jk} = 1, \qquad j = 1, \ldots, N.$

The capacity of each vehicle cannot be exceeded: $\sum_{j=1}^{N} d_j x_{jk} \leq Q, \qquad k = 1, \ldots, K.$

The objective uses pairwise customer distances to encourage coherent clusters. Route optimization within each cluster is a separate subsequent step.

Three QUBO Formulations

1. Slack Variables

$$\left(Q - \sum_j d_j x_{jk} - s_k\right)^2$$

Explicitly encode unused capacity.

Cost: additional binary variables.

2. Unbalanced Penalty

Define

$$h_k = Q - \sum_j d_j x_{jk}.$$

Then add

$$-\lambda_1 h_k + \lambda_2 h_k^2.$$

Benefit: no capacity slack bits.

3. Variable Reduction

Normalize capacity:

$$\sum_j \frac{d_j}{Q} x_{jk} \leq 1.$$

Use binary slack variables for the normalized constraint.

Goal: reduce slack-bit count.

All formulations also need a penalty enforcing the assignment constraints, such as

$$\beta \sum_j \left(\sum_k x_{jk} - 1\right)^2.$$

Mapping the QUBO to Pasqal Hardware

The neutral-atom device evolves under the Hamiltonian

$$\frac{H(t)}{\hbar} = \frac{\Omega(t)}{2} \sum_i \hat{\sigma}_i^x - \delta(t) \sum_i \hat{n}_i + \sum_{i<j} \frac{C_6}{\|r_i - r_j\|^6} \hat{n}_i \hat{n}_j.$$

- Each binary variable is mapped to one atom.
- Detuning $\delta(t)$ encodes linear QUBO coefficients.
- Position-dependent interactions encode quadratic couplings.

Why variable reduction matters

Slack variables also require atoms. Eliminating them can allow larger clustering instances to fit within the hardware's atom budget.

However, fitting the variable count does not guarantee that the required couplings can be implemented faithfully with a particular atom geometry.

Main Result: Sample Solution

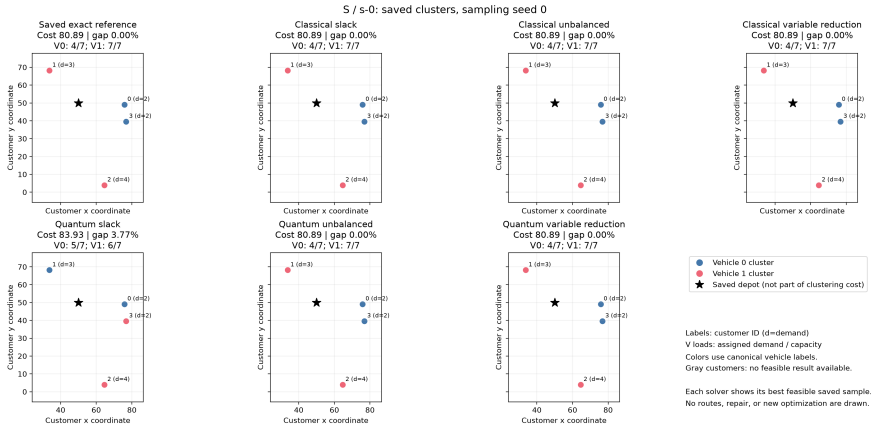


Figure: Comparison of classical and quantum solvers of the clustering problem for a sample instance of S-Tier Problem with the demand $d_j = (2, 3, 4, 2)$. The tuning parameters for the Unbalanced Penalization were $\lambda_1 = 0.2$, $\lambda_2 = 0.01$.

Main Result: Fewer Variables

Encoding	Tier S	Tier M	Tier L
Assignment only / unbalanced	8	30	60
Variable reduction	10	33	64
Logarithmic slack	14	45	80
Linear slack	22	90	180

80-atom count budget

- Unbalanced penalties: all three tiers fit by count.
- Linear slack: M and L exceed the budget.
- Logarithmic slack: Tier L uses all 80 atoms.

Theoretical variable counts reported in the document. Each binary variable corresponds to one atom in the mapping. These counts do not establish hardware embedding feasibility.

Experimental Results and Open Challenges

Experiments

The document compares classical and quantum solver results for three tiers.

- S: 4 customers;
- M: 10 customers;
- L: 15 customers.

Observed limitation

Some M- and L-tier runs did not produce feasible bitstrings.

Next steps

The document identifies several directions:

- Tune penalty parameters λ_1, λ_2;
- Investigate device coupling strength;
- Explore different atom-grid geometries.

Important distinction

A larger classical–quantum energy gap alone does not establish that a model fails. Feasibility of the returned bitstring is an essential practical criterion.

Conclusion

1. Capacity-constrained clustering requires both unique assignments and feasible vehicle loads.
2. The unbalanced-penalty formulation eliminates capacity slack variables.
3. The resulting variable counts are

   $$\boxed{8,\ 30,\ 60}$$

   for S, M, and L, respectively.
4. Feasible solutions for the larger instances remain a challenge requiring further penalty and hardware tuning.

Fewer variables create room for scaling,
but reliable solutions still require careful tuning.